

Task 6: ML Project Deployment Guide

A Step-by-Step Guide for Streamlit and React + FastAPI Projects

Covers: Streamlit Community Cloud • Vercel (Frontend) • Render (Backend)

Before You Start

This guide shows you how to take a finished project and put it on a live website that anyone can open with a link. It covers two common situations:

- Part 1 — Your project is a Streamlit app (a single Python app, ML model included).
- Part 2 — Your project has a React frontend and a FastAPI backend (two separate pieces that talk to each other).

Requirements before you begin:

- A free GitHub account, with your project pushed to a GitHub repository.
- A requirements.txt file listing your Python packages (for Streamlit or FastAPI apps).
- A package.json file for your React app (created automatically by Create React App / Vite).

Tip: Always push your latest code to GitHub before deploying. All three platforms below deploy directly from a GitHub repository, and they automatically redeploy every time you push new commits.

Part 1: Deploying a Streamlit App (Streamlit Community Cloud)

Use this if your entire project — including your machine learning model — runs inside a single Streamlit app (one Python file that calls `st.something`).

Step-by-step

1. Push your project to GitHub. Make sure the repository contains your main app file (commonly `streamlit_app.py` or `app.py`) and a `requirements.txt` file listing every package you import (e.g. `streamlit`, `scikit-learn`, `pandas`).
2. Go to share.streamlit.io in your browser.
3. Sign in with your GitHub account and authorize Streamlit to access your repositories (public repos work without extra permissions; for private repos, grant the additional access Streamlit asks for).
4. Click "Create app" (or "New app") from your Streamlit Community Cloud workspace.
5. Fill in the deployment form: choose your repository, the branch (usually `main`), and the path to your app file (e.g. `streamlit_app.py`). You can also paste a direct GitHub URL to the file instead.
6. Optional: under "Advanced settings," pick your Python version and add any Secrets your app needs (API keys, tokens) in TOML format — never commit secrets directly to GitHub.
7. Optional: set a custom subdomain in the "App URL" field, e.g. `your-app-name`, so your link becomes `https://your-app-name.streamlit.app`.
8. Click "Deploy." Streamlit Community Cloud builds a container, installs your `requirements.txt` packages, and starts your app. This usually takes 1–5 minutes.
9. Once deployed, your app is live at a `streamlit.app` URL that you can share with anyone.
10. To update the live app later, simply push new commits to the same GitHub branch — Streamlit Community Cloud detects the change and redeploys automatically within a minute or two.

Common issues

- App won't build: check `requirements.txt` for typos, and pin package versions if a newer version breaks compatibility.
- Large ML model files: don't commit files over ~100 MB to GitHub directly; load them from a cloud storage link, or use Git LFS.
- App sleeps after inactivity on the free tier: this is normal — it wakes up automatically when someone visits the link.

Part 2: Deploying a React + FastAPI Project

Use this if your project has two separate pieces: a React frontend (the UI) and a FastAPI backend (the API / ML model server). These two pieces are deployed to two different platforms and then connected together.

- Frontend (React) → Vercel
- Backend (FastAPI) → Render

Step A: Deploy the FastAPI Backend to Render

Deploy the backend first, so you have its live URL ready when you configure the frontend.

1. Make sure your backend folder contains: your FastAPI app (e.g. main.py with an `app = FastAPI()` object), a requirements.txt listing fastapi, uvicorn, and any other libraries you import (e.g. scikit-learn, torch), and that the whole project is pushed to GitHub.
2. In your FastAPI code, enable CORS so the frontend domain is allowed to call the API:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://your-frontend.vercel.app"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

3. Go to render.com and sign in (or sign up) with your GitHub account.
4. From the dashboard, click "New" → "Web Service."
5. Connect the GitHub repository containing your FastAPI backend, and give Render permission to access it.
6. Configure the service:

Environment: Python 3

Build Command: `pip install -r requirements.txt`

Start Command: `uvicorn main:app --host 0.0.0.0 --port $PORT` (replace main:app with the correct module:variable name for your app)

Instance Type: Free

7. Add any environment variables your app needs (API keys, database URLs) under the "Environment" tab.
8. Click "Create Web Service." Render installs your dependencies and starts the server — this takes a few minutes on the first deploy.
9. Once live, copy your backend's URL, shown in the format `https://your-service-name.onrender.com` — you'll need this for the frontend.

Tip: Render's free tier spins the service down after ~15 minutes of no traffic and takes a few seconds to "wake up" on the next request. This is normal for free hosting and fine for student projects.

Step B: Deploy the React Frontend to Vercel

1. In your React code, make sure API calls point to an environment variable rather than a hardcoded localhost address, e.g.:

```
const API_URL = import.meta.env.VITE_API_URL; // Vite
// or: process.env.REACT_APP_API_URL for Create React App

fetch(`${API_URL}/predict`, { method: "POST", ... });
```

2. Push your frontend folder to GitHub (it can be the same repo as the backend, in a separate subfolder, or its own repo).
3. Go to vercel.com and sign in with your GitHub account.
4. Click "Add New..." → "Project."

5. Import the GitHub repository containing your React app. If your frontend is in a subfolder (e.g. /frontend), set that as the "Root Directory" in the project settings.
6. Vercel auto-detects your framework (Create React App, Vite, Next.js, etc.) and fills in the Build Command and Output Directory. Leave these as detected unless your setup is custom.
7. Add an environment variable pointing to your live backend, e.g. VITE_API_URL = https://your-service-name.onrender.com (use the exact variable name your code reads).
8. Click "Deploy." Vercel builds and deploys your app, usually in under a minute.
9. Once live, your app is available at a URL like https://your-project.vercel.app — this is the link you share.
10. Go back to your FastAPI CORS settings on Render and make sure allow_origins includes this exact Vercel URL, then redeploy the backend if you changed it.

Step C: Connect and Test

1. Open your live Vercel URL in a browser.
2. Use the app as a normal visitor would, and confirm it successfully calls your Render backend (check the browser's Network tab in Developer Tools if something doesn't load).
3. If requests fail with a CORS error, double-check that the allow_origins list on Render exactly matches your Vercel domain (including https://, no trailing slash).
4. To update either app later, just push new commits to GitHub — Vercel and Render both redeploy automatically.

Quick Reference

Project Type	Platform	Where to Start	Deploys From
Streamlit app	Streamlit Community Cloud	share.streamlit.io	GitHub repo
React frontend	Vercel	vercel.com	GitHub repo
FastAPI backend	Render	render.com	GitHub repo

Golden rule for all three platforms:

- Push to GitHub → the platform builds automatically → you get a public link → push again anytime to update.